

# SNOWFLAKE

## End-to-End ETL Project

Retail Analytics Platform — Medallion Architecture

AWS S3 (CSV) → Bronze → Silver → Gold → BI

Covers: Storage Integration · File Formats · Stages · COPY INTO · Snowpipe  
Streams · Tasks · DAGs · SCD Type 2 · RBAC · Masking · Row Policies  
UDFs · Stored Procedures · Resource Monitors · Time Travel · Data Quality

CHAPTER 0

# Project Overview & Architecture

This project implements a production-grade Snowflake ETL pipeline for a retail company. CSV files arrive in AWS S3 daily and are processed through three layers using the Medallion architecture pattern.

## Business Context

A retail company receives daily sales data from their order management system as CSV files in AWS S3. The data engineering team needs to build a reliable, scalable pipeline that transforms raw data into business-ready analytics for their BI team.

## Architecture — Medallion (Bronze → Silver → Gold)

| Layer  | Database  | Purpose                                 | Key Operations                           |
|--------|-----------|-----------------------------------------|------------------------------------------|
| Bronze | BRONZE_DB | Raw landing zone — exact copy of source | COPY INTO, Snowpipe AUTO_INGEST, Streams |
| Silver | SILVER_DB | Cleansed, typed, validated, SCD2 dims   | Stored procs, MERGE, SCD2, validation    |
| Gold   | GOLD_DB   | Business-ready aggregations & marts     | Aggregations, RFM, Materialized Views    |
| Common | COMMON_DB | Shared objects across all layers        | Stages, file formats, UDFs, audit logs   |

## Source Data — CSV Files in S3

| File            | S3 Path          | Grain                 | Frequency   | Key Columns                                       |
|-----------------|------------------|-----------------------|-------------|---------------------------------------------------|
| orders.csv      | raw/orders/      | One row per order     | Daily       | order_id, customer_id, total_amount, order_date   |
| customers.csv   | raw/customers/   | One row per customer  | Daily delta | customer_id, name, email, segment, region         |
| products.csv    | raw/products/    | One row per product   | Weekly      | product_id, name, category, unit_cost, unit_price |
| order_items.csv | raw/order_items/ | One row per line item | Daily       | item_id, order_id, product_id, qty, line_total    |
| regions.csv     | raw/regions/     | One row per region    | Monthly     | region_id, region_name, country, territory        |

## Snowflake Objects Created

| Object Type                | Count | Names                                                                                                                     |
|----------------------------|-------|---------------------------------------------------------------------------------------------------------------------------|
| Databases                  | 4     | BRONZE_DB, SILVER_DB, GOLD_DB, COMMON_DB                                                                                  |
| Schemas                    | 13    | RAW, ARCHIVE, DIMENSIONS, FACTS, STAGING, SALES_MART, CUSTOMER_MART, PRODUCT_MART, REPORTING, INGESTION, UTILITIES, AUDIT |
| Virtual Warehouses         | 5     | INGESTION_WH, TRANSFORM_WH, ANALYTICS_WH, DATA_SCIENCE_WH, DEV_WH                                                         |
| Custom Roles               | 8     | ETL_ADMIN, DATA_ENGINEER, ETL_ROLE, SENIOR_ANALYST, ANALYST, JUNIOR_ANALYST, DATA_SCIENTIST, REPORT_VIEWER                |
| Tables                     | 12+   | 5 Bronze raw, 5 Silver (dims+facts), Gold marts, Audit log                                                                |
| Stages                     | 6     | STG_S3_ORDERS, CUSTOMERS, PRODUCTS, ORDER_ITEMS, REGIONS, ARCHIVE                                                         |
| Pipes (Snowpipe)           | 3     | PIPE_ORDERS, PIPE_CUSTOMERS, PIPE_ORDER_ITEMS                                                                             |
| Streams                    | 4     | STREAM_RAW_ORDERS, CUSTOMERS, PRODUCTS, DIM_CUSTOMERS                                                                     |
| Tasks (DAG)                | 9     | ROOT + 8 child tasks — full pipeline DAG                                                                                  |
| Stored Procedures          | 8+    | SP_LOAD_DIM_*, SP_LOAD_FACT_*, SP_BUILD_GOLD_*, SP_RUN_DQ_CHECKS                                                          |
| Masking Policies           | 4     | MASK_EMAIL, MASK_PHONE, MASK_CREDIT_LIMIT, MASK_CUSTOMER_NAME                                                             |
| Row Access Policies        | 1     | POLICY_TERRITORY_ACCESS                                                                                                   |
| UDFs/UDTFs                 | 7     | CLEAN_PHONE, ORDER_TIER, BUSINESS_DAYS, FISCAL_YEAR, ROW_HASH, DATE_SERIES, SPLIT_TAGS                                    |
| Resource Monitors          | 4     | RM_ACCOUNT_MONTHLY, RM_ANALYTICS, RM_TRANSFORM, RM_DEV_WEEKLY                                                             |
| Views (incl. Materialized) | 4     | MV_SALES_KPI_DAILY, V_SALES_DASHBOARD, V_SALES_TREND + secure views                                                       |

## Setup — Databases, Warehouses & RBAC

## 1.1 Virtual Warehouse Strategy

| Warehouse       | Size                       | Type               | Auto-suspend | Used By                                             |
|-----------------|----------------------------|--------------------|--------------|-----------------------------------------------------|
| INGESTION_WH    | X-Small                    | Standard           | 60s          | COPY INTO, Snowpipe — I/O bound, small size is fine |
| TRANSFORM_WH    | Medium                     | Standard           | 120s         | Bronze→Silver→Gold ELT, Streams & Tasks             |
| ANALYTICS_WH    | Medium (multi-cluster 1-3) | Standard           | 180s         | BI tools, analyst queries — scales for concurrency  |
| DATA_SCIENCE_WH | Large                      | Snowpark-Optimized | 300s         | Python/ML workloads — 16x memory per node           |
| DEV_WH          | X-Small                    | Standard           | 30s          | Developer sandbox — aggressive suspend              |

```
-- =====
-- FILE: 01_setup/01_databases_warehouses.sql
-- PURPOSE: Create all databases, schemas, and virtual warehouses
-- RUN AS: SYSADMIN
-- =====

USE ROLE SYSADMIN;

-- 
-- DATABASES — Medallion Architecture
-- 

-- BRONZE: Raw landing zone — exact copy of source, no transforms
CREATE DATABASE IF NOT EXISTS BRONZE_DB
    DATA_RETENTION_TIME_IN_DAYS = 1    -- transient-like: can recreate from S3
    COMMENT = 'Raw landing zone — unmodified source data from S3';

-- SILVER: Cleansed, typed, validated, conformed
```

## 1.2 RBAC Role Hierarchy

Roles are organized in a hierarchy so each tier inherits privileges from the tier below. Senior roles can do everything junior roles can do, plus more.

| Role                | Warehouse Access    | Bronze        | Silver        | Gold              | Purpose                 |
|---------------------|---------------------|---------------|---------------|-------------------|-------------------------|
| ETL_ADMIN_ROLE      | INGESTION+TRANSFORM | ALL           | ALL           | ALL               | Pipeline admin          |
| DATA_ENGINEER_ROLE  | TRANSFORM+DEV       | SELECT+DML    | ALL           | ALL               | ETL development         |
| ETL_ROLE            | INGESTION+TRANSFORM | SELECT+INSERT | INSERT+UPDATE | ALL               | Service account         |
| SENIOR_ANALYST_ROLE | ANALYTICS           | None          | SELECT        | SELECT            | Power users             |
| ANALYST_ROLE        | ANALYTICS           | None          | None          | SELECT views      | Business analysts       |
| JUNIOR_ANALYST_ROLE | ANALYTICS           | None          | None          | SELECT (masked)   | Read-only reporting     |
| REPORT_VIEWER_ROLE  | ANALYTICS           | None          | None          | SELECT views only | BI tool service account |

**KEY CONCEPT** The four required grants for ANY user to run a query: USAGE on DATABASE + USAGE on SCHEMA + SEL

## 01\_setup/02\_rbac\_roles\_grants.sql

```
-- =====
-- FILE: 01_setup/02_rbac_roles_grants.sql
-- PURPOSE: Create role hierarchy, grant privileges
-- RUN AS: SECURITYADMIN (for roles) + SYSADMIN (for object grants)
-- =====

-- ████████████████████████████████████████████████████████████████████████████
-- ROLE HIERARCHY DIAGRAM
-- ████████████████████████████████████████████████████████████████████████████
--
-- ACCOUNTADMIN
--   █
-- SECURITYADMIN ██████ SYSADMIN
--   █               █
-- USERADMIN       ETL_ADMIN_ROLE
--                   █
--                 ████████████████████████████████████████████████████████████
-- DATA_ENGINEER ANALYST_ROLE DATA_SCIENTIST_ROLE
```





## 2.2 COPY INTO vs Snowpipe — When to Use Each

| Aspect         | COPY INTO (Batch)                          | Snowpipe (Continuous)              |
|----------------|--------------------------------------------|------------------------------------|
| Trigger        | Manual or scheduled task                   | S3 event notification (automatic)  |
| Latency        | Minutes to hours (depends on schedule)     | 1–5 minutes from file arrival      |
| Billing        | Warehouse credits per execution            | Serverless — per file processed    |
| Best for       | Historical loads, backfills, large batches | Near-real-time, daily file drops   |
| Dedup          | 64-day load history (ETag-based)           | 64-day load history (ETag-based)   |
| Error handling | ON_ERROR = CONTINUE/ABORT/SKIP_FILE        | Errors logged, pipeline continues  |
| This project   | Used for initial load + backfill           | Used for ongoing daily order files |

## 02\_bronze/02\_copy\_into\_snowpipe.sql

```
USE ROLE ETL_ROLE;  
USE WAREHOUSE INGESTION_WH;  
USE DATABASE BRONZE_DB;  
USE SCHEMA RAW;
```

```
--
-- SECTION A: MANUAL / SCHEDULED BATCH COPY INTO
-- Use this for: initial historical loads, backfills, reruns
--
```

```
-- ■■■ INFER SCHEMA (run once to understand file structure) ■■■■■■
-- Validate file structure before committing to a load

SELECT * FROM TABLE(
  INFER_SCHEMA(
    LOCATION      => '@COMMON_DB.INGESTION.STG_S3_ORDERS',
    FILE_FORMAT   => 'COMMON_DB.INGESTION.CSV_STANDARD_FMT'
  )
);
```

```
-- Preview raw data from stage (no warehouse needed)
SELECT $1, $2, $3, $4, $5
FROM @COMMON_DB.INGESTION.STG_S3_ORDERS
(FILE_FORMAT => 'COMMON_DB.INGESTION.CSV_STANDARD_FMT')
LIMIT 10;
```

```
-- ■■ VALIDATE before loading (dry run, no data inserted) ■■■■■■
COPY INTO BRONZE_DB.RAW.RAW_ORDERS (
    ORDER_ID, CUSTOMER_ID, ORDER_DATE, SHIP_DATE, STATUS,
    SHIP_MODE, REGION_ID, TOTAL_AMOUNT, DISCOUNT_PCT, PROFIT,
    LOAD FILE
```

## CHAPTER 3

## Silver Layer — Cleansed & Conformed

Silver applies business rules: type casting, deduplication, FK resolution, and SCD Type 2 for dimension tables. This is where raw becomes reliable.

### 3.1 SCD Type 2 Design

SCD Type 2 preserves full history by inserting a new row for each attribute change rather than overwriting. The pattern uses `EFF_START_DATE`, `EFF_END_DATE`, and `IS_CURRENT`.

| Column         | Type                    | Purpose                                            |
|----------------|-------------------------|----------------------------------------------------|
| SK_CUSTOMER    | NUMBER<br>AUTOINCREMENT | Surrogate key — unique per version, never reused   |
| CUSTOMER_ID    | VARCHAR                 | Natural/business key — same across all versions    |
| EFF_START_DATE | DATE                    | When this version became active                    |
| EFF_END_DATE   | DATE (nullable)         | When this version was superseded (NULL = current)  |
| IS_CURRENT     | BOOLEAN                 | TRUE for the live record — always filter with this |

**NOTE** SCD2 interview trap: when a customer changes their email, you must: 1) Expire the old row (set EFF\_END\_DATE and

## 03\_silver/01\_silver\_tables.sql — Table Definitions

```
USE ROLE DATA_ENGINEER_ROLE;  
USE WAREHOUSE TRANSFORM_WH;  
USE DATABASE SILVER DB;
```

```
-- ██████████  
-- DIM_CUSTOMERS – SCD Type 2  
-- Tracks full history of customer attribute changes  
-- ██████████
```

USE SCHEMA DIMENSIONS:

```
CREATE TABLE IF NOT EXISTS DIM_CUSTOMERS (
    -- Surrogate key (generated, never from source)
```

## 03\_silver/02\_scd2\_procedures.sql — SCD2 Stored Procedures

### 3.2 Silver Fact Table Design

Fact tables in Silver reference dimension surrogate keys. Foreign key resolution happens in the stored procedure using JOIN to DIM tables with IS\_CURRENT = TRUE to get the current surrogate key.

| Fact Table            | Grain                 | Dimensions Joined                                        | Key Metrics                                            |
|-----------------------|-----------------------|----------------------------------------------------------|--------------------------------------------------------|
| FACT_ORDERS           | One row per order     | DIM_CUSTOMERS,<br>DIM_REGIONS, DIM_DATE                  | TOTAL_AMOUNT,<br>DISCOUNT_PCT,<br>PROFIT, SHIP_MODE    |
| FACT_ORDER_ITEM<br>MS | One row per line item | DIM_CUSTOMERS,<br>DIM_PRODUCTS,<br>DIM_REGIONS, DIM_DATE | QUANTITY,<br>UNIT_PRICE,<br>LINE_TOTAL,<br>RETURN_FLAG |

CHAPTER 4

# Gold Layer — Business-Ready Analytics

Gold pre-aggregates data into mart tables and views optimized for BI tool consumption. RFM scoring, Customer 360, and materialized views ensure sub-second dashboard queries.

## 4.1 Gold Layer Objects

| Object                 | Type              | Refresh           | Business Purpose                                            |
|------------------------|-------------------|-------------------|-------------------------------------------------------------|
| DAILY_SALES_SUMMARY    | Table             | Hourly (task)     | Pre-aggregated daily KPIs by region — fast dashboard source |
| MONTHLY_CATEGORY_SALES | Table             | Daily (task)      | Product category performance by month                       |
| CUSTOMER_360           | Table             | Daily (task)      | Full customer profile with RFM scores and CLV estimate      |
| MV_SALES_KPI_DAILY     | Materialized View | Auto (background) | Real-time daily KPIs — Snowflake auto-refreshes             |
| V_SALES_DASHBOARD      | Secure View       | N/A (query MV)    | BI-safe view of KPIs — no PII, no internal metrics          |
| V_SALES_TREND          | Secure View       | N/A               | Rolling averages, day-over-day change, region ranking       |

## 4.2 RFM Scoring — Customer Segmentation

RFM (Recency, Frequency, Monetary) scoring assigns 1-5 scores to each customer across three dimensions using NTILE window functions. The combined score maps to Platinum/Gold/Silver/Bronze customer tiers.

RFM Scoring Pattern

```

-- RFM scoring using NTILE window functions
WITH RFM_SCORES AS (
  SELECT SK_CUSTOMER,
    -- Recency: score 5=bought recently, 1=bought long ago
    NTILE(5) OVER (ORDER BY DAYS_SINCE_LAST_ORDER ASC) AS R_SCORE,
    -- Frequency: score 5=many orders, 1=few orders
    NTILE(5) OVER (ORDER BY TOTAL_ORDERS DESC) AS F_SCORE,
    -- Monetary: score 5=high spend, 1=low spend
    NTILE(5) OVER (ORDER BY TOTAL_REVENUE DESC) AS M_SCORE
  FROM ORDER_METRICS
)
SELECT SK_CUSTOMER,
  R_SCORE, F_SCORE, M_SCORE,
  R_SCORE + F_SCORE + M_SCORE AS TOTAL_RFM,

```

04\_gold/01\_gold\_tables\_views.sql — Gold Layer

## CHAPTER 5

# Streams, Tasks & Pipeline DAG

Streams track changes in Bronze tables. Tasks form a DAG that automatically processes new data through all three layers. The pipeline is fully event-driven.

## 5.1 Stream Types Used in This Project

| Stream               | Type        | On Table                           | Why This Type                                       |
|----------------------|-------------|------------------------------------|-----------------------------------------------------|
| STREAM_RAW_ORDERS    | Append-only | BRONZE_DB.RAW.RAW_ORDERS           | Bronze tables only receive INSERTs — more efficient |
| STREAM_RAW_CUSTOMERS | Append-only | BRONZE_DB.RAW.RAW_CUSTOMERS        | Same — Bronze is insert-only from COPY INTO         |
| STREAM_RAW_PRODUCTS  | Append-only | BRONZE_DB.RAW.RAW_PRODUCTS         | Same                                                |
| STREAM_DIM_CUSTOMERS | Standard    | SILVER_DB.DIMENSIONS.DIM_CUSTOMERS | SCD2 generates INSERTS and UPDATES — need full CDC  |

**KEY CONCEPT** Stream offset only advances when a DML transaction that **CONSUMES** the stream commits. Plain SELECTs do not.

## 5.2 Task DAG Structure

| Task                     | Type             | Dependency          | WHEN Guard                           |
|--------------------------|------------------|---------------------|--------------------------------------|
| TASK_PIPELINE_ROOT       | Root (scheduler) | None — has SCHEDULE | STREAM_HAS_DATA on any Bronze stream |
| TASK_LOAD_DIM_REGIONS    | Child            | AFTER ROOT          | None — runs if root runs             |
| TASK_LOAD_DIM_CUSTOMERS  | Child (parallel) | AFTER ROOT          | None                                 |
| TASK_LOAD_DIM_PRODUCTS   | Child (parallel) | AFTER ROOT          | None                                 |
| TASK_LOAD_FACT_ORDERS    | Child            | AFTER all 3 dims    | None — waits for all dims            |
| TASK_RUN_DQ_CHECKS       | Child            | AFTER FACT_ORDERS   | Data quality gate before Gold        |
| TASK_BUILD_DAILY_SUMMARY | Child            | AFTER FACT_ORDERS   | None                                 |

|                         |                  |                       |                        |
|-------------------------|------------------|-----------------------|------------------------|
| TASK_BUILD_CUSTOMER_360 | Child (parallel) | AFTER FACT_ORDERS     | None                   |
| TASK_NOTIFY_SUCCESS     | Leaf             | AFTER both Gold tasks | None — logs completion |

**NOTE CRITICAL:** Resume tasks LEAF-FIRST, ROOT-LAST. Resuming the root task before children are resumed will cause

## 05\_pipelines/01\_streams\_tasks\_dag.sql

```
-- =====
-- FILE: 05_pipelines/01_streams_tasks_dag.sql
-- PURPOSE: Streams for CDC + Task DAG for full pipeline
-- PIPELINE: Bronze → Silver → Gold (automated end-to-end)
-- RUN AS: DATA_ENGINEER_ROLE
-- =====

USE ROLE DATA_ENGINEER_ROLE;
USE WAREHOUSE TRANSFORM_WH;

-- ████████████████████████████████████████████████████████████████████████████

-- SECTION A: STREAMS on Bronze tables
-- Track new rows arriving from Snowpipe / COPY INTO
-- ████████████████████████████████████████████████████████████████████████████

-- Append-only streams – Bronze tables are insert-only (never updated)
CREATE STREAM IF NOT EXISTS BRONZE_DB.RAW.STREAM_RAW_ORDERS
    ON TABLE BRONZE_DB.RAW.RAW_ORDERS
    APPEND_ONLY = TRUE           -- more efficient for insert-only source
    COMMENT = 'CDC stream on raw orders – detects new rows from COPY INTO/Snowpipe';

CREATE STREAM IF NOT EXISTS BRONZE_DB.RAW.STREAM_RAW_CUSTOMERS
    ON TABLE BRONZE_DB.RAW.RAW_CUSTOMERS
    APPEND_ONLY = TRUE
    COMMENT = 'CDC stream on raw customers';

CREATE STREAM IF NOT EXISTS BRONZE_DB.RAW.STREAM_RAW_PRODUCTS
    ON TABLE BRONZE_DB.RAW.RAW_PRODUCTS
    APPEND_ONLY = TRUE
    COMMENT = 'CDC stream on raw products';

-- Standard stream on Silver dimensions (tracks SCD2 changes)
CREATE STREAM IF NOT EXISTS SILVER_DB.DIMENSIONS.STREAM_DIM_CUSTOMERS
    ON TABLE SILVER_DB.DIMENSIONS.DIM_CUSTOMERS
    COMMENT = 'CDC stream on Silver dim_customers – feeds Gold customer mart';

-- Check stream health
SELECT SYSTEM$STREAM_HAS_DATA('BRONZE_DB.RAW.STREAM_RAW_ORDERS') AS has_orders;
SELECT SYSTEM$STREAM_HAS_DATA('BRONZE_DB.RAW.STREAM_RAW_CUSTOMERS') AS has_customers;

-- ████████████████████████████████████████████████████████████████████████████

-- SECTION B: TASK DAG – Full pipeline orchestration
--
-- DAG structure:
--
-- TASK_ROOT (scheduler, every 1hr)
--   └─
--       ├── TASK_LOAD_SILVER_DIMS (parallel)
--       ├──     ├── TASK_LOAD_DIM_REGIONS
--       └──     └─ TASK_LOAD_DIM_CUSTOMERS
```

## CHAPTER 6

## Security & Governance

Four layers of security: RBAC controls object access, masking policies hide sensitive column values, row access policies filter rows by territory, and network policies restrict connections by IP.

## 6.1 Masking Policies Applied

| Column                  | Policy             | ETL/Admin sees  | Senior Analyst sees | Analyst sees       |
|-------------------------|--------------------|-----------------|---------------------|--------------------|
| customers.EMAIL         | MASK_EMAIL         | full@email.com  | fu***@domain.com    | ***@domain.com     |
| customers.PHONE         | MASK_PHONE         | +1-555-867-5309 | ***-***-5309        | ***-***-5309       |
| customers.CREDIT_LIMIT  | MASK_CREDIT_LIMIT  | 47500           | 47000 (rounded)     | NULL               |
| customers.CUSTOMER_NAME | MASK_CUSTOMER_NAME | Alice Johnson   | Alice Johnson       | Customer-a3f2b1... |

## 6.2 Row Access Policy — Territory Filtering

Sales representatives can only see data from their assigned territories. The policy uses a mapping table — adding access = one INSERT, no DDL changes needed.

## 06\_security/01\_security\_policies.sql

```
-- =====
-- FILE: 06_security/01_security_policies.sql
-- PURPOSE: Dynamic masking, row access, network policies
-- RUN AS: SECURITYADMIN (policies) + SYSADMIN (apply to objects)
-- =====

USE ROLE SECURITYADMIN;
USE DATABASE SILVER_DB;

-- 
-- SECTION A: DYNAMIC DATA MASKING POLICIES
-- 

-- Email masking: full for ETL/admin, partial for analysts, masked for others
CREATE OR REPLACE MASKING POLICY SILVER_DB.DIMENSIONS.MASK_EMAIL
AS (val VARCHAR) RETURNS VARCHAR ->
CASE
    WHEN CURRENT_ROLE() IN ('DATA_ENGINEER_ROLE', 'ETL_ADMIN_ROLE', 'ACCOUNTADMIN')
    THEN val
    WHEN CURRENT_ROLE() = 'SENIOR_ANALYST_ROLE'
    THEN REGEXP_REPLACE(val, '^(^@){1,2})([^@]*)(@.*)', '\\1***\\3')
    ELSE
        CONCAT('***@', SPLIT_PART(val, '@', 2))
END
```

## 6.3 Object Tagging for Data Classification

PII columns are tagged with `TAG_PII='email'|'phone'|'name'|'financial'`. Tables are tagged with `TAG_SENSITIVITY='public'|'internal'|'confidential'|'restricted'`. These tags drive masking policy recommendations and compliance reporting.

CHAPTER 7

# Monitoring, Resource Monitors & Cost Management

Resource monitors cap credit usage. ACCOUNT\_USAGE views provide pipeline health, cost attribution, and security auditing.

## 7.1 Resource Monitor Hierarchy

| Monitor              | Applied To     | Quota        | Frequency | Actions                                                  |
|----------------------|----------------|--------------|-----------|----------------------------------------------------------|
| RM_ACCOUNT_MONTHLY   | Entire account | 5000 credits | Monthly   | Notify @50/75/90%, Suspend @100%, SuspendImmediate @115% |
| RM_ANALYTICS_MONTHLY | ANALYTICS_WH   | 1500 credits | Monthly   | Notify @80%, Suspend @100%                               |
| RM_TRANSFORM_MONTHLY | TRANSFORM_WH   | 2000 credits | Monthly   | Notify @80%, Suspend @100%                               |
| RM_DEV_WEEKLY        | DEV_WH         | 50 credits   | Weekly    | Notify @90%, SuspendImmediate @100%                      |

## 7.2 Key ACCOUNT\_USAGE Views for This Project

| View                       | Latency | Used For                                               |
|----------------------------|---------|--------------------------------------------------------|
| WAREHOUSE_METERING_HISTORY | ~45 min | Credit usage per warehouse — chargeback reporting      |
| QUERY_HISTORY              | ~45 min | Find slow queries, expensive operations, top users     |
| ACCESS_HISTORY             | ~3 hrs  | Column-level audit — who queried PII columns           |
| LOGIN_HISTORY              | ~2 hrs  | Failed login alerts, unauthorized access detection     |
| METERING_DAILY_HISTORY     | ~45 min | Daily credit trend — cost spike detection              |
| TABLE_STORAGE_METRICS      | ~2 hrs  | Storage cost by table — Time Travel and Fail-safe size |

```
07_monitoring/01_resource_monitors_alerts.sql

-- =====
-- FILE: 07_monitoring/01_resource_monitors_alerts.sql
-- PURPOSE: Resource monitors, cost tracking, ACCOUNT_USAGE monitoring
-- RUN AS: ACCOUNTADMIN
-- =====
```

## CHAPTER 8

# Stored Procedures, UDFs & Advanced Functions

The project uses four stored procedure languages: SQL Scripting for ETL logic, JavaScript for dynamic DDL, Python for data profiling, and Python UDFs for vectorized column transformations.

## 8.1 UDFs Created

| UDF Name                      | Language            | Returns | Purpose                                             |
|-------------------------------|---------------------|---------|-----------------------------------------------------|
| UDF_CLEAN_PHONE               | SQL                 | VARCHAR | Standardize phone to +1-XXX-XXX-XXXX                |
| UDF_ORDER_TIER                | SQL                 | VARCHAR | Classify order: Micro/Small/Medium/Large/Enterprise |
| UDF_BUSINESS_DAYS             | SQL                 | NUMBER  | Count Mon-Fri days between two dates                |
| UDF_FISCAL_YEAR               | SQL                 | VARCHAR | Return FY string for Apr-Mar fiscal year            |
| UDF_ROW_HASH                  | SQL                 | VARCHAR | MD5 hash of 4 columns for change detection          |
| UDF_CLASSIFY_CUSTOMER_SEGMENT | Python (vectorized) | VARCHAR | Pandas-based segment classifier from order metrics  |
| UDF_DATE_SERIES (UDTF)        | SQL                 | TABLE   | Generate one row per date between two dates         |
| UDF_SPLIT_TAGS (UDTF)         | SQL                 | TABLE   | Split delimited string into rows                    |

## 08 stored procedures/01 udfs advanced procs.sql

```
-- =====
-- FILE: 08_stored_procedures/01_udfs_advanced_procs.sql
-- PURPOSE: UDFs, Python stored procedures, utility functions
-- RUN AS: DATA_ENGINEER_ROLE
-- =====

USE ROLE DATA_ENGINEER_ROLE;
USE WAREHOUSE TRANSFORM_WH;
USE DATABASE COMMON_DB;
USE SCHEMA UTILITIES;

-- =====
-- SECTION A: SCALAR UDFs (SQL)
-- =====

-- Clean and standardize phone numbers
CREATE OR REPLACE FUNCTION UDF_CLEAN_PHONE(phone_raw VARCHAR)
```

# Data Quality Framework

## 9.1 DQ Checks Implemented

[illegible]

## CHAPTER 10

# Execution Guide & Project Summary

Step-by-step execution order for deploying the project from scratch.

## 10.1 Deployment Sequence

| Step | File                                             | Role                     | Purpose                                                      |
|------|--------------------------------------------------|--------------------------|--------------------------------------------------------------|
| 1    | 01_setup/01_databases_warehouses.sql             | SYSADMIN                 | Create databases, schemas, warehouses                        |
| 2    | 01_setup/02_rbac_roles_grants.sql                | SECURITYADMIN + SYSADMIN | Create roles, grants, users                                  |
| 3    | 01_setup/03_integrations_stages_file_formats.sql | ACCOUNTADMIN + SYSADMIN  | Storage integration, stages, file formats, sequences, SNS    |
| 4    | 02_bronze/01_bronze_tables.sql                   | DATA_ENGINEER_ROLE       | Create all Bronze transient tables                           |
| 5    | 02_bronze/02_copy_into_snowpipe.sql              | ETL_ROLE                 | Initial COPY INTO + create Snowpipes                         |
| 6    | 03_silver/01_silver_tables.sql                   | DATA_ENGINEER_ROLE       | Create SCD2 dims, fact tables, date dimension                |
| 7    | 03_silver/02_scd2_procedures.sql                 | DATA_ENGINEER_ROLE       | Create all Silver stored procedures                          |
| 8    | 04_gold/01_gold_tables_views.sql                 | DATA_ENGINEER_ROLE       | Create Gold marts, materialized views, secure views          |
| 9    | 08_stored_procedures/01_udfs_advanced_procs.sql  | DATA_ENGINEER_ROLE       | Create all UDFs, UDTFs, Python procs                         |
| 10   | 09_data_quality/01_dq_checks.sql                 | DATA_ENGINEER_ROLE       | Create DQ procedure and audit tables                         |
| 11   | 06_security/01_security_policies.sql             | SECURITYADMIN + SYSADMIN | Create and apply masking, row access, network policies, tags |
| 12   | 07_monitoring/01_resource_monitors_alerts.sql    | ACCOUNTADMIN             | Create resource monitors, apply to warehouses                |
| 13   | 05_pipelines/01_streams_tasks_dag.sql            | DATA_ENGINEER_ROLE       | Create streams, task DAG, resume all tasks                   |

## 10.2 Topics Covered in This Project

| Snowflake Topic | Where Used | File |
|-----------------|------------|------|
|-----------------|------------|------|

|                                   |                                                       |                         |
|-----------------------------------|-------------------------------------------------------|-------------------------|
| Storage Integration (AWS S3)      | S3 → Snowflake connection without credentials         | 01_setup/03             |
| File Formats (CSV, JSON, Parquet) | CSV_STANDARD_FMT, CSV_PIPE_FMT, JSON_FMT, PARQUET_FMT | 01_setup/03             |
| External Stages                   | STG_S3_ORDERS, CUSTOMERS, PRODUCTS, etc.              | 01_setup/03             |
| COPY INTO (load)                  | Bronze layer initial + batch loads                    | 02_bronze/02            |
| COPY INTO (unload)                | Export pattern for reverse pipeline                   | 02_bronze/02            |
| INFER_SCHEMA / AUTO_DETECT        | Validate file structure before loading                | 02_bronze/02            |
| Snowpipe AUTO_INGEST              | PIPE_ORDERS, PIPE_CUSTOMERS, PIPE_ORDER_ITEMS         | 02_bronze/02            |
| Transient Tables                  | All 5 Bronze raw tables                               | 02_bronze/01            |
| Permanent Tables                  | All Silver dims, facts, Gold marts                    | 03_silver/01            |
| External Tables                   | ext_orders example for S3 querying                    | 03_silver/01            |
| Dynamic Tables                    | ORDERS_SUMMARY example                                | 03_silver/01            |
| SCD Type 2                        | DIM_CUSTOMERS, DIM_PRODUCTS full implementation       | 03_silver/02            |
| Time Travel                       | SP_CREATE_SNAPSHOT uses AT clause                     | 08_stored_procedures/01 |
| Zero-Copy Cloning                 | SP_CREATE_SNAPSHOT creates instant table clone        | 08_stored_procedures/01 |
| Streams (Append-only)             | STREAM_RAW_ORDERS/CUSTOMERS/PRODUCTS                  | 05_pipelines/01         |
| Streams (Standard)                | STREAM_DIM_CUSTOMERS for full CDC                     | 05_pipelines/01         |
| Tasks (Standalone)                | TASK_PIPELINE_ROOT with CRON schedule                 | 05_pipelines/01         |
| Tasks (DAG)                       | 9-task pipeline DAG with parallel branches            | 05_pipelines/01         |
| Serverless Tasks                  | SERVERLESS_CLEANUP example                            | 05_pipelines/01         |
| RBAC Roles & Grants               | 8 custom roles, full hierarchy                        | 01_setup/02             |
| Dynamic Data Masking              | 4 policies on email, phone, credit, name              | 06_security/01          |
| Row Access Policies               | Territory-based row filtering with mapping table      | 06_security/01          |
| Network Policies                  | NP_CORPORATE_ACCESS, NP_ETL_SERVICE_ONLY              | 06_security/01          |

|                           |                                                        |                             |
|---------------------------|--------------------------------------------------------|-----------------------------|
| Object Tagging            | TAG_PII, TAG_SENSITIVITY on columns & tables           | 06_security/01              |
| Secure Data Sharing       | Secure views required for sharing pattern              | 06_security/01              |
| Resource Monitors         | 4 monitors — account + 3 per-warehouse                 | 07_monitoring/01            |
| Stored Procs (SQL)        | SP_LOAD_DIM_*,<br>SP_LOAD_FACT_*,<br>SP_BUILD_GOLD_*   | 03_silver/02,<br>04_gold/01 |
| Stored Procs (JavaScript) | SP_ARCHIVE_OLD_DATA — dynamic DDL                      | 08_stored_procedures/01     |
| Stored Procs (Python)     | SP_PROFILE_TABLE — pandas profiling                    | 08_stored_procedures/01     |
| Scalar UDFs               | UDF_CLEAN_PHONE, ORDER_TIER, BUSINESS_DAYS, etc.       | 08_stored_procedures/01     |
| Vectorized Python UDF     | UDF_CLASSIFY_CUSTOMER_SEGMENT — pandas Series          | 08_stored_procedures/01     |
| UDTFs                     | UDF_DATE_SERIES,<br>UDF_SPLIT_TAGS                     | 08_stored_procedures/01     |
| Standard Views            | Internal reporting views                               | 04_gold/01                  |
| Secure Views              | V_SALES_DASHBOARD,<br>V_SALES_TREND — BI access        | 04_gold/01                  |
| Materialized Views        | MV_SALES_KPI_DAILY — auto-refreshed KPIs               | 04_gold/01                  |
| Clustering Keys           | FACT_ORDERS by (ORDER_DATE_KEY, SK_CUSTOMER)           | 03_silver/01                |
| Sequences                 | SEQ_CUSTOMER_SK etc. for surrogate keys                | 01_setup/03                 |
| Window Functions          | NTILE for RFM, LAG for day-over-day, RANK for region   | 04_gold/01                  |
| ACCOUNT_USAGE             | Metering, query history, access history, login history | 07_monitoring/01            |
| SNS Task Notifications    | ERROR_INTEGRATION on all tasks + child notify task     | 05_pipelines/01             |
| Data Quality Framework    | 6 DQ checks with PASS/WARN/FAIL + trend logging        | 09_data_quality/01          |

**KEY CONCEPT** This project covers 40+ distinct Snowflake features across all layers. Every file is production-ready and fo